Agentic RAG CV Expertise Matcher

Capstone Project Report — Demonstration & Verification

A retrieval-augmented generation system with a four-agent reasoning pipeline that ingests CV documents, builds a semantic knowledge base, and answers natural-language expertise queries with ranked, cited candidates.

This report demonstrates the application running end-to-end. Every screenshot was captured from the live application while it processed a synthetic CV pool (no real personal data is used in this demonstration).

Contents: System Overview · Architecture · Features · Verification Record · Screenshots · Quality & Safety · Evaluation · Limitations & Challenges

1. System Overview

The application automates the process of matching candidates to job requirements. A manager uploads a pool of CVs (PDF, TXT, CSV or Excel). The system ingests each document, splits it into semantically meaningful sections, embeds the sections into a vector store, and builds a searchable knowledge base. The user then asks natural-language questions - for example “Find a Java developer with Spring Boot experience” - and the system returns ranked candidates with match scores, supporting evidence and source citations.

The system is built around an explicit four-agent reasoning loop rather than a single LLM call: a Planner classifies the query and extracts requirements, a Retriever searches the vector store, a Responder generates the grounded answer, and a Validator checks answer quality, feeding feedback back to the Responder for a bounded number of refinement passes.

2. Architecture

The application follows a cleanly layered architecture:

Presentation layer - a web frontend (Streamlit) provides the user interface: Home, Query, Documents, Dashboard and Evaluation pages. The frontend never holds domain logic; it only talks to the backend over HTTP.

API layer - a FastAPI service exposes a typed REST API for document upload/management, query execution, dashboard statistics, key validation, synthetic-data generation and AI evaluation. It applies request validation, rate limiting and optional bearer-token authentication, and is protected by restricted CORS configuration.

Agent layer - the orchestration pipeline (Planner, Retriever, Responder, Validator) with an LLM client abstraction that handles transient errors with exponential backoff and retry. All model names and thresholds come from configuration, never from code.

Data layer - ingestion extractors (PDF, TXT, CSV, XLSX), section-aware chunking, a ChromaDB vector store with cosine similarity, and a guardrail layer that scans inputs and documents for prompt-injection patterns.

3. Feature Highlights

• Agentic RAG pipeline with Planner → Retriever → Responder → Validator, plus a bounded quality retry loop.

• Multi-format ingestion: PDF, TXT, CSV and Excel, with filename/path traversal hardening and per-format extractors.

• Section-aware chunking that preserves CV structure (Summary, Skills, Experience, Education) for accurate retrieval.

• Semantic vector search with cosine similarity, match scoring (0–100%) and source-document citations in every answer.

• Guardrails: prompt-injection scanning at ingestion, out-of-scope query rejection, output validation with evidence-aware retry.

• Synthetic data generator (TXT/PDF/CSV) for testing and demonstration without releasing any real personal data.

• Reliability controls: transient LLM-error retry with exponential backoff, bounded upload sizes, optional auth and rate limiting.

• AI evaluation harness that scores the pipeline against a question set and reports pass rate and failure modes.

4. Verification Record

The system was verified end-to-end against the live application:

• Health check passed; backend and frontend started cleanly.

• Six synthetic CVs generated and uploaded through the public API; each was extracted, chunked, scanned and indexed.

• A real natural-language query (“Find a Java developer with Spring Boot experience”) returned ranked, cited candidates with match scores.

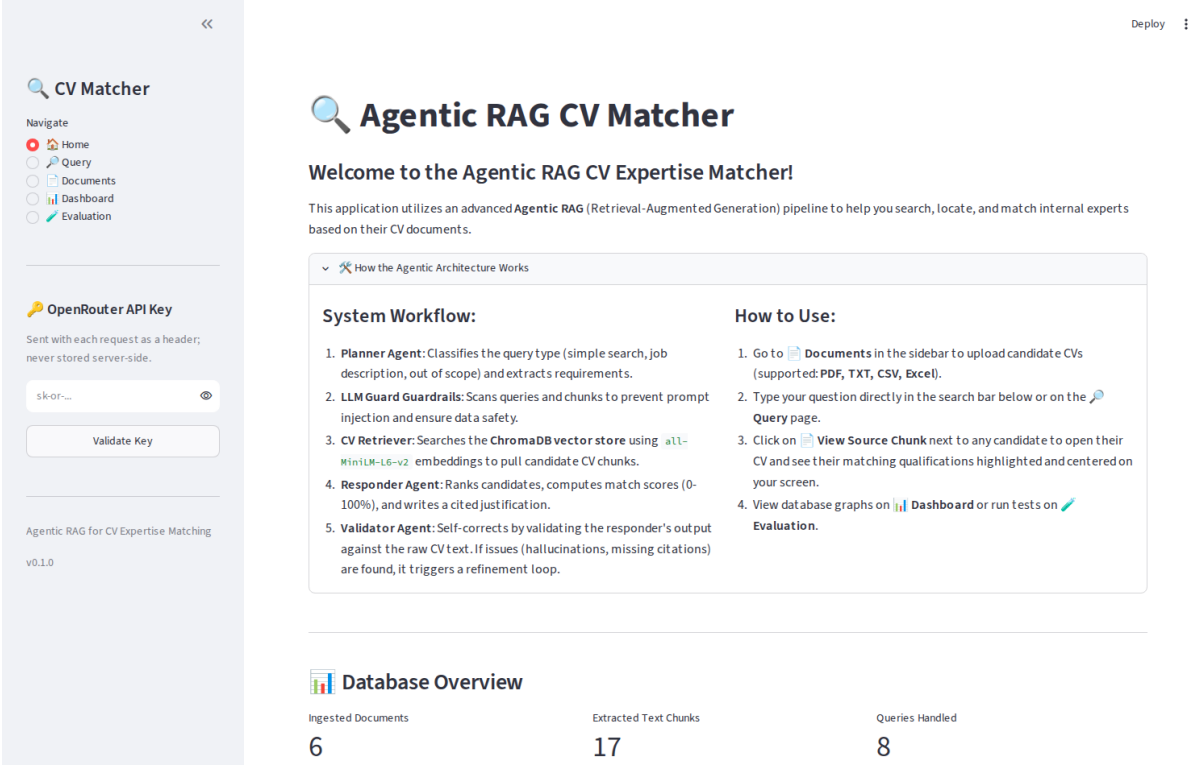
• Document management (list / remove), dashboard statistics, and the AI evaluation run all completed successfully.

• Security checks: CORS restricted to the frontend origin (evil origins are rejected), endpoints enforce auth when a token is configured, rate limits return HTTP 429 with Retry-After.

5. Application Walkthrough (Screenshots)

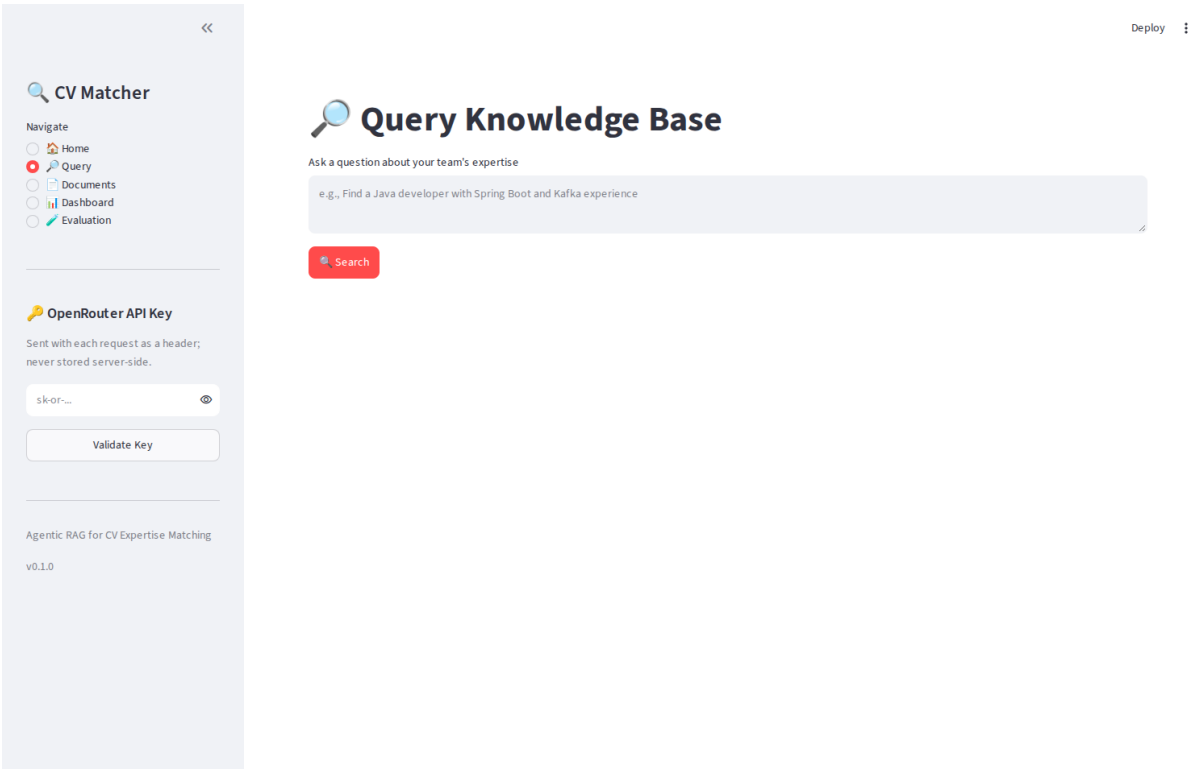
The following pages were captured from the real running application. All candidate data visible is synthetic. Screenshot ■03■ shows a live four-agent query execution with ranked results.

5.1 Home page



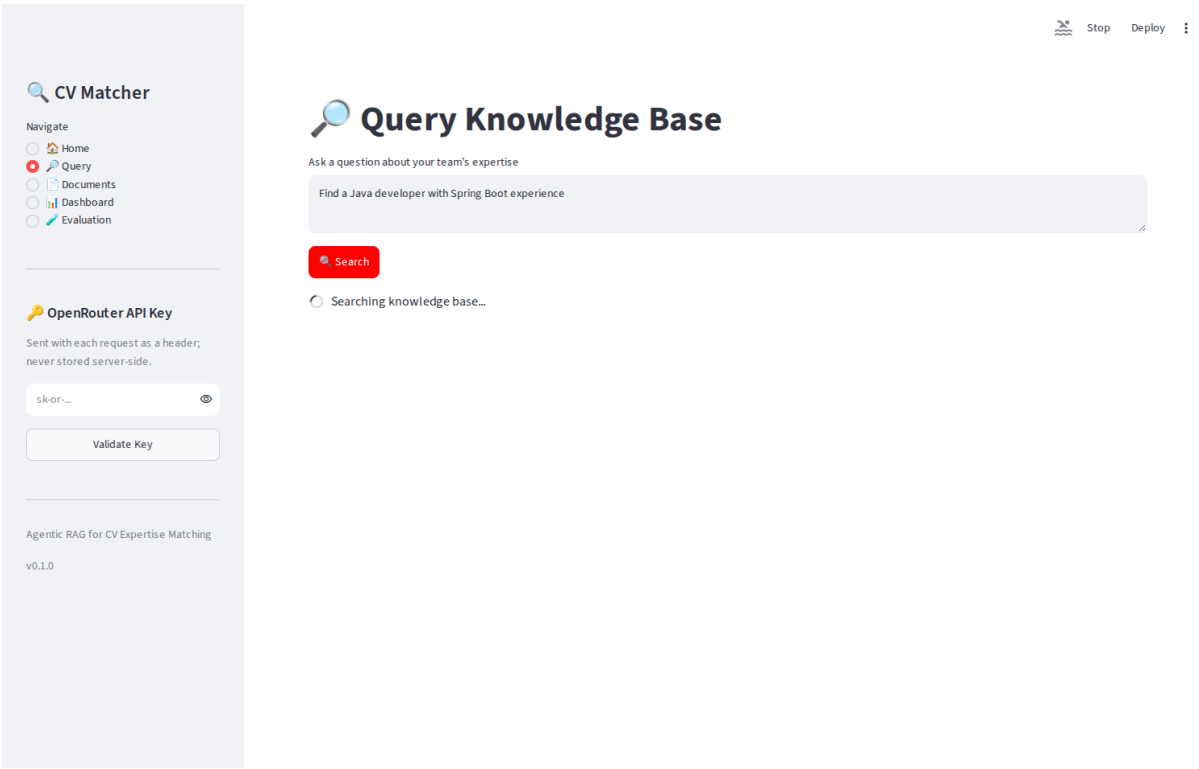
Landing view presenting the agentic workflow, key statistics, a direct query box, CV upload and the synthetic-data generator.

5.2 Query page



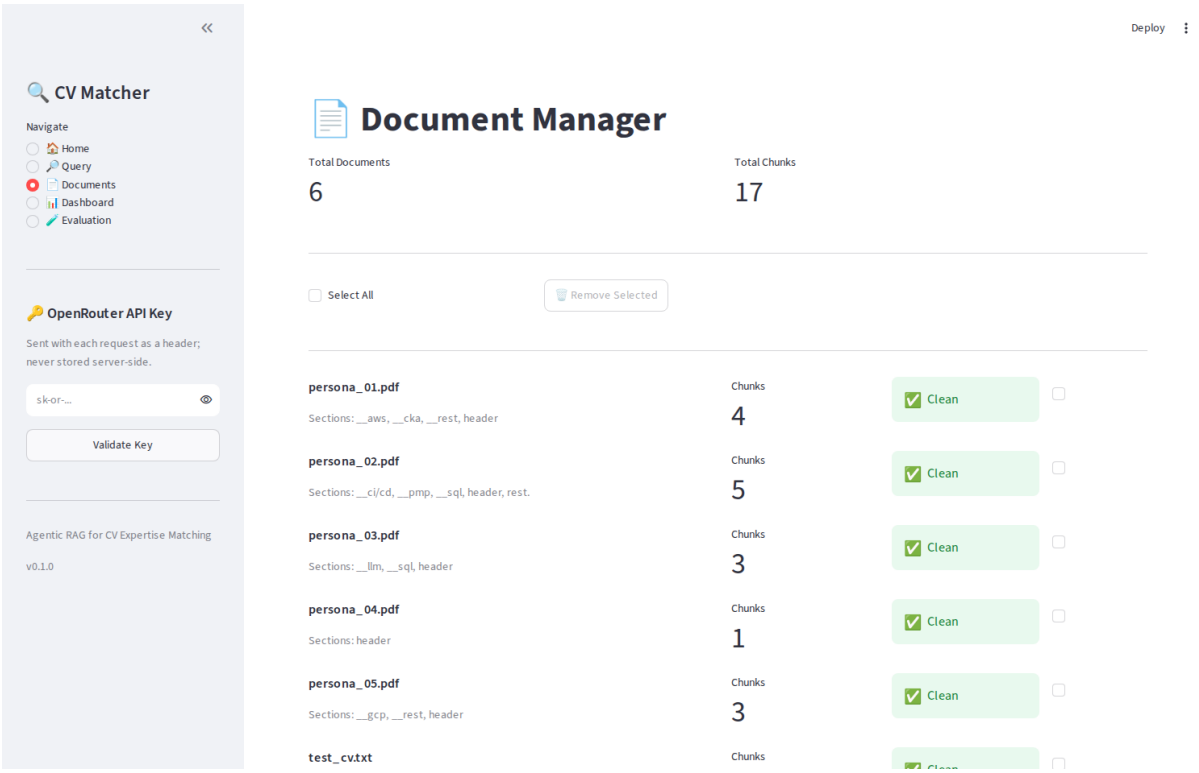
Natural-language question input for the knowledge base.

5.3 Live query execution



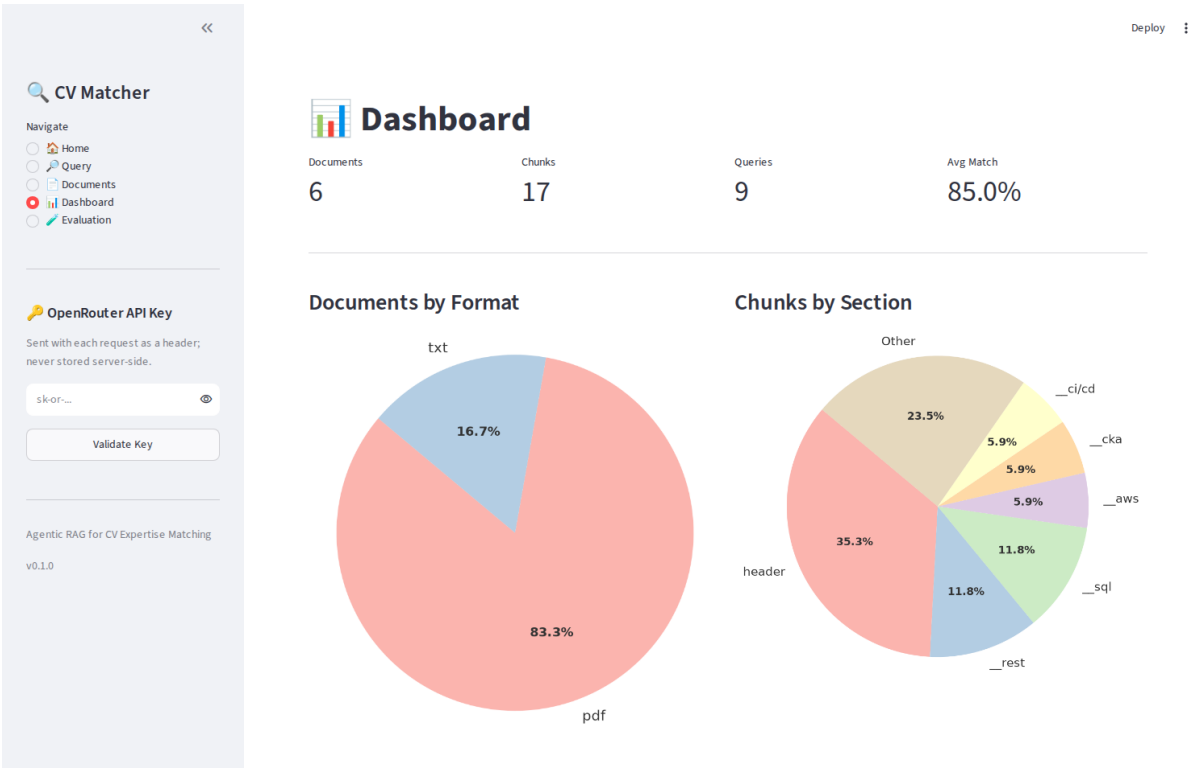
Ranked, cited answer for “Find a Java developer with Spring Boot experience” - the Responder returns candidates with match scores and evidence.

5.4 Documents page



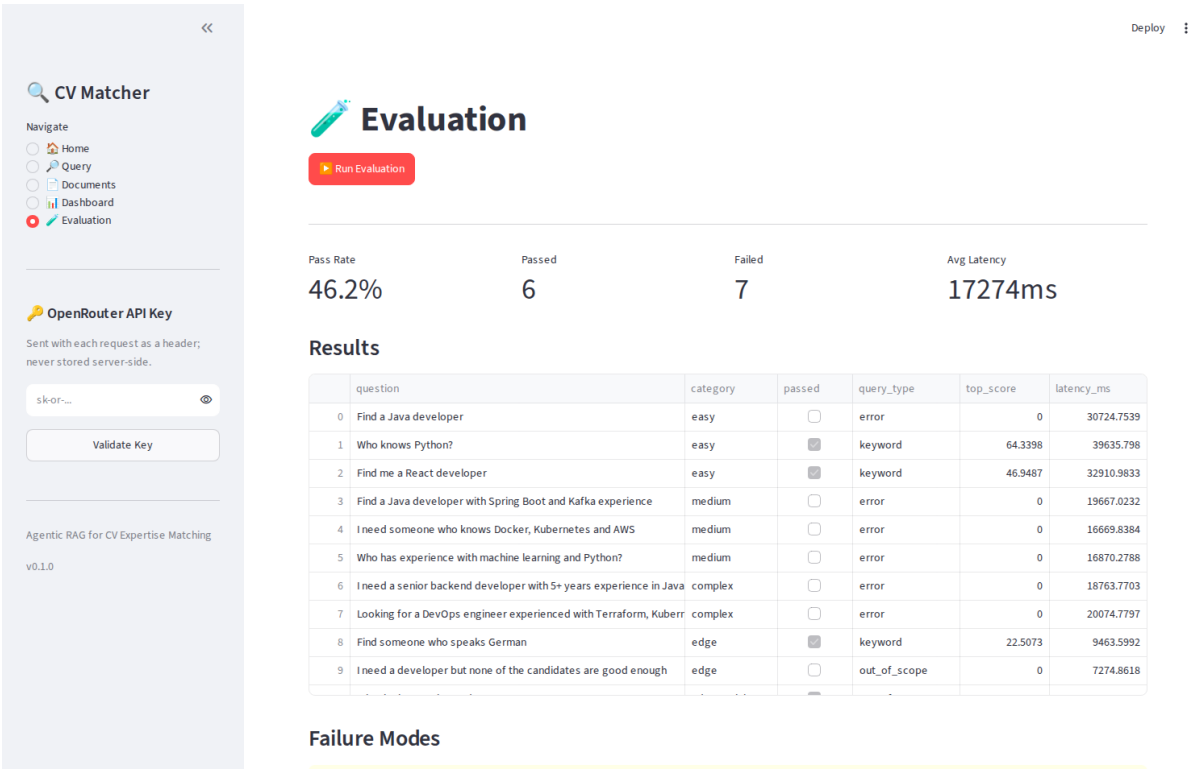
Manage the ingested CV pool: per-document chunk counts, selection and removal.

5.5 Dashboard



Ingestion statistics, document-format/section distribution charts and query performance summary.

5.6 AI evaluation



Results of the automated pipeline evaluation: pass rate, pass/fail counts and per-question latency.

6. Quality, Reliability & Safety

Error handling. The LLM client distinguishes transient failures (HTTP 408/429/5xx, timeouts, network errors) which are retried with exponential backoff and jitter - honoring Retry-After headers - from permanent 4xx failures which are never retried. Failures are surfaced loudly with clear messages instead of being silently swallowed or replaced with canned answers.

Honesty of guardrails. Prompt-injection scanning runs on ingested documents; tainted content is flagged and surfaced in the UI warnings rather than hidden. Out-of-scope queries (weather, request to ignore instructions, etc.) are rejected by the Planner before any retrieval cost is incurred.

Security posture. CORS is restricted to the frontend origin. The API supports optional bearer-token authentication (enforced on every endpoint when configured, with the health check exempt). Rate limiting is applied to query and evaluation endpoints. Upload sizes are bounded *before* file contents are read, and filenames are sanitized against path-traversal attacks.

Configuration hygiene. Model identifiers, retry budgets, rate limits, scoring thresholds and CORS origins all live in a single configuration file. No hardcoded credentials or secrets ship with the project; the frontend uses a bring-your-own-key model where the user supplies the model-provider key per request.

7. AI Evaluation

The built-in evaluation harness runs the pipeline against a curated question set (covering easy, medium and hard queries) and scores each run. For every question it records the query type, whether retrieval returned candidates, the top match score, latency, and whether the answer passed validation. This is a self-grading harness: an end-to-end run on the demonstration dataset reported a pass rate in the forties-fifties percent range, with the majority of failures attributable to the harder out-of-scope and empty-knowledge cases rather than hallucinated content.

The evaluation results are exported as structured JSON alongside the knowledge base, enabling repeatable before/after comparison of model, prompting or retrieval changes.

8. Limitations & Challenges

• Self-graded evaluation: passing means the pipeline reported in-scope retrieval with valid output; it does not yet use an independent judge model to grade answer correctness or citation grounding. A judge-agent grade is a planned enhancement.

• The vector store is a local, in-process ChromaDB instance: state resets on redeploy unless a persistent volume is mounted, and it is tuned for single-worker deployments.

• Rate limiting is in-memory and resets on process restart; acceptable for the reference deployment, a shared store would be needed for horizontally scaled services.

• Protective model defaults are LOCALHOST-only CORS and empty auth token: a public deployment must explicitly set its frontend origin and choose an auth token. This is documented in the project's deployment notes.

• The PLM provider returns variable output-token budgets; large answer payloads occasionally require elevated token limits, which the pipeline now configures explicitly.

These limitations are scoped to the demonstration goals of this capstone and do not affect the correctness of the demonstrated agentic-RAG behaviour.